

bomb lab

x86-64에서 함수의 calling convention은 다음과 같다.

- 함수의 인자 : rdi -> rsi -> rdx -> rcx -> r8 -> r9 -> stack
- 함수의 리턴값 : rax

gdb를 사용할 때, Intel syntax를 사용하여 분석했다.

phase_1

```
Dump of assembler code for function phase_1:
0x000000000400ef0 <+0>:  sub    rsp,0x8
0x000000000400ef4 <+4>:  mov    esi,0x40255c
0x000000000400ef9 <+9>:  call   0x4012fe <strings_not_equal>
0x000000000400efe <+14>:  test   eax,eax
0x000000000400f00 <+16>:  je     0x400f07 <phase_1+23>
0x000000000400f02 <+18>:  call   0x401564 <explode_bomb>
0x000000000400f07 <+23>:  add    rsp,0x8
0x000000000400f0b <+27>:  ret
```

phase_1. esi에 담긴 "0x40255c"이 의심스럽고, <strings_not_equal> 함수 실행 이후, eax의 값이 존재하면, 정상적으로 통과할 수 있게 된다.

```
gdb> x/s 0x40255c
0x40255c: "Public speaking is very easy."
```

<strings_not_equal> 의 esi 인자로 들어가는 0x40255c 메모리를 읽으면, "Public speaking is very easy." 라는 문자열이 있는 것을 확인 할 수 있다.

- rdi : (우리가 입력한 문자열)
- rsi(esi) : "Public speaking is very easy."

```
Dump of assembler code for function strings_not_equal:
0x0000000004012fe <+0>:  push   r12
0x000000000401300 <+2>:  push   rbp
0x000000000401301 <+3>:  push   rbx
0x000000000401302 <+4>:  mov    rbx,rdi
0x000000000401305 <+7>:  mov    rbp,rsi
0x000000000401308 <+10>: call   0x4012e1 <string_length>      // 사용자 입력 길이
0x00000000040130d <+15>: mov    r12d,eax
0x000000000401310 <+18>: mov    rdi,rbp
0x000000000401313 <+21>: call   0x4012e1 <string_length>      // 원래 있던 문자열
0x000000000401318 <+26>: mov    edx,0x1
0x00000000040131d <+31>: cmp    r12d,eax // 길이 비교
0x000000000401320 <+34>: jne    0x401360 <strings_not_equal+98> // 다르면
0x000000000401322 <+36>: movzx  eax,BYTE PTR [rbx]
0x000000000401325 <+39>: test   al,al // nullbyte 판단
0x000000000401327 <+41>: je     0x40134d <strings_not_equal+79>
0x000000000401329 <+43>: cmp    al,BYTE PTR [rbp+0x0]
0x00000000040132c <+46>: je     0x401337 <strings_not_equal+57>
0x00000000040132e <+48>: xchg   ax,ax // ?
0x000000000401330 <+50>: jmp    0x401354 <strings_not_equal+86>
0x000000000401332 <+52>: cmp    al,BYTE PTR [rbp+0x0]
0x000000000401335 <+55>: jne    0x40135b <strings_not_equal+93>
0x000000000401337 <+57>: add    rbx,0x1
0x00000000040133b <+61>: add    rbp,0x1
0x00000000040133f <+65>: movzx  eax,BYTE PTR [rbx]
0x000000000401342 <+68>: test   al,al
0x000000000401344 <+70>: jne    0x401332 <strings_not_equal+52>
0x000000000401346 <+72>: mov    edx,0x0
0x00000000040134b <+77>: jmp    0x401360 <strings_not_equal+98> // 종료
0x00000000040134d <+79>: mov    edx,0x0
0x000000000401352 <+84>: jmp    0x401360 <strings_not_equal+98>
0x000000000401354 <+86>: mov    edx,0x1
0x000000000401359 <+91>: jmp    0x401360 <strings_not_equal+98>
0x00000000040135b <+93>: mov    edx,0x1
0x000000000401360 <+98>: mov    eax,edx
0x000000000401362 <+100>: pop    rbx
0x000000000401363 <+101>: pop    rbp
0x000000000401364 <+102>: pop    r12
0x000000000401366 <+104>: ret
```

길이가 같은 경우, 루프를 돌게 된다. rdi -> rbx -> eax, rsi -> rbp로, 문자열이 이동하고, eax와 rbp의 한 바이트가 같은지 판단한다. 같으면 포인터 값을 1 증가시켜서 루프를 돌고, 다른 경우 종료한다. 루프를 모두 돌고 종료하면 eax에 1이 들어가고, 그렇지 않다면 0이 들어가게 된다. 즉 이 함수는 한 바이트씩 문자열을 비교해 같은지 판단하는 함수라고 볼 수 있다.

```
Dump of assembler code for function string_length:
0x00000000004012e1 <+0>:    cmp     BYTE PTR [rdi],0x0
0x00000000004012e4 <+3>:    je      0x4012f8 <string_length+23>
0x00000000004012e6 <+5>:    mov     rdx,rdi
0x00000000004012e9 <+8>:    add     rdx,0x1
0x00000000004012ed <+12>:   mov     eax,edx
0x00000000004012ef <+14>:   sub     eax,edi
0x00000000004012f1 <+16>:   cmp     BYTE PTR [rdx],0x0
0x00000000004012f4 <+19>:   jne     0x4012e9 <string_length+8>
0x00000000004012f6 <+21>:   repz    ret
0x00000000004012f8 <+23>:   mov     eax,0x0
0x00000000004012fd <+28>:   ret
```

rdx의 1바이트 값을 읽어와서 0x0과 값을 비교해서, 아니면 루프를 돌아 rdx 값을 1 증가시킨다. 증가시키기 전의 값이 rdi에 들어 있으므로, 루프를 돈 이후, eax에 +1이 저장된다. 0x0을 만나면 루프가 종료하므로, 이 함수는 rdi의 레지스터가 가리키는 메모리의 문자열의 길이를 eax에 저장하는 함수다.

- eax : rdi와 rsi의 문자열이 같으면 1, 다르면 0

즉, 아래의 입력을 입력하면 통과할 수 있다.

```
0x40255c:      "Public speaking is very easy."
```

phase_2

phase_2에 들어 있는 <read_six_numbers> 함수를 먼저 살펴보자.

```
Dump of assembler code for function read_six_numbers:
0x000000000040164e <+0>:    sub     rsp,0x18
0x0000000000401652 <+4>:    mov     rdx,rsi
0x0000000000401655 <+7>:    lea     rcx,[rsi+0x4]
0x0000000000401659 <+11>:   lea     rax,[rsi+0x14]
0x000000000040165d <+15>:   mov     QWORD PTR [rsp+0x8],rax
0x0000000000401662 <+20>:   lea     rax,[rsi+0x10]
0x0000000000401666 <+24>:   mov     QWORD PTR [rsp],rax
0x000000000040166a <+28>:   lea     r9,[rsi+0xc]
0x000000000040166e <+32>:   lea     r8,[rsi+0x8]
0x0000000000401672 <+36>:   mov     esi,0x4032f0
0x0000000000401677 <+41>:   mov     eax,0x0
0x000000000040167c <+46>:   call    0x400c30 <__isoc99_sscanf@plt>    // eax는 읽은 원소의 개수가 저장된다.
0x0000000000401681 <+51>:   cmp     eax,0x5                          // if eax > 5
0x0000000000401684 <+54>:   jg      0x40168b <read_six_numbers+61>
0x0000000000401686 <+56>:   call    0x401564 <explode_bomb>
...
```

```
gdb> x/s 0x4032f0
0x4032f0:      "%d %d %d %d %d %d"
```

sscanf의 인자로 들어가는 esi 레지스터의 값을 확인해보니, 6개의 숫자를 인자로 받는 것을 알 수 있었다.

sscanf 이후에 어떻게 값이 설정되는지 알기 위해 breakpoint를 걸어서 확인해보자.

```
>> b *0x0000000000401681
>> c
1 2 3 4 5 6
...
$rax 0x0 -> 0x6

|stack|
0x00007fffffff870|+0x0020: 0x0000000020000001
0x00007fffffff878|+0x0028: 0x0000000040000003
0x00007fffffff880|+0x0030: 0x0000000060000005
```

확인 결과, rax에는 받은 인자의 개수가 리턴되고, stack에 입력한 숫자 6개가 들어간 것을 확인 할 수 있었다. <read_six_numbers>+61에서 "add rsp, 0x18" 명령어를 사용하므로, 첫번째 입력값은 현재 [rsp + 0x20]에서 [rsp + 0x8]으로 이동하고, 6번째 입력값은 [rsp + 0x38]에서 [rsp + 0x20]이 될 것이다.

```
Dump of assembler code for function phase_2:
0x0000000000400f0c <+0>:    push    rbp
0x0000000000400f0d <+1>:    push    rbx
```

```

0x0000000000400f0e <+2>: sub    rsp,0x28
0x0000000000400f12 <+6>: mov    rsi,rsp
0x0000000000400f15 <+9>: call   0x40164e <read_six_numbers> // 6개의 숫자 입력을 받음
0x0000000000400f1a <+14>: cmp    DWORD PTR [rsp],0x1 // 1번째 입력값이 1
0x0000000000400f1e <+18>: je     0x400f40 <phase_2+52>
0x0000000000400f20 <+20>: call   0x401564 <explode_bomb>
0x0000000000400f25 <+25>: jmp    0x400f40 <phase_2+52>
0x0000000000400f27 <+27>: mov    eax,DWORD PTR [rbx-0x4]
0x0000000000400f2a <+30>: add    eax,eax // eax = 2 * eax
0x0000000000400f2c <+32>: cmp    DWORD PTR [rbx],eax
0x0000000000400f2e <+34>: je     0x400f35 <phase_2+41>
0x0000000000400f30 <+36>: call   0x401564 <explode_bomb>
0x0000000000400f35 <+41>: add    rbx,0x4 // rbx += 4
0x0000000000400f39 <+45>: cmp    rbx,rbp
0x0000000000400f3c <+48>: jne    0x400f27 <phase_2+27>
0x0000000000400f3e <+50>: jmp    0x400f4c <phase_2+64>
0x0000000000400f40 <+52>: lea    rbx,[rsp+0x4] // 1번째 입력값
0x0000000000400f45 <+57>: lea    rbp,[rsp+0x18] // 6번째 입력값
0x0000000000400f4a <+62>: jmp    0x400f27 <phase_2+27>
...

```

첫번째 값 [rsp]를 1과 비교, 첫번째 입력 값이 1이어야한다. [rsp+0x4] -> rbx, [rsp + 0x18] -> rbp로 지정된다. 여기서 [rsp+0x4]는 우리가 입력했던 두번째 값이고, [rsp + 0x18]는 6번째 값이다. 그 후, [rbx-0x4] (첫번째 입력값) -> eax로 이동하고, add eax, eax 즉 2배를 계산한다. 그 후, [rbx] (두번째 입력값) 과 비교한다. 이런식으로 루프가 진행된다. 즉, phase_2에서는 1부터 시작해 2배 증가하는 수를 6개 입력하면 통과할 수 있을 것이다.

1 2 4 8 16 32

phase_3

```

Dump of assembler code for function phase_3:
0x0000000000400f53 <+0>: sub    rsp,0x18
0x0000000000400f57 <+4>: lea    rcx,[rsp+0x8]
0x0000000000400f5c <+9>: lea    rdx,[rsp+0xc]
0x0000000000400f61 <+14>: mov    esi,0x4032fc
0x0000000000400f66 <+19>: mov    eax,0x0
0x0000000000400f6b <+24>: call   0x400c30 <__isoc99_sscanf@plt>
0x0000000000400f70 <+29>: cmp    eax,0x1 // eax > 1
0x0000000000400f73 <+32>: jg     0x400f7a <phase_3+39>
0x0000000000400f75 <+34>: call   0x401564 <explode_bomb>
0x0000000000400f7a <+39>: cmp    DWORD PTR [rsp+0xc],0x7 // 처음 입력값이 7보다 크면 터짐
0x0000000000400f7f <+44>: ja     0x400fe5 <phase_3+146>
0x0000000000400f81 <+46>: mov    eax,DWORD PTR [rsp+0xc] // 입력값
0x0000000000400f85 <+50>: jmp    QWORD PTR [rax*8+0x402590] // 이동하는 구문
0x0000000000400f8c <+57>: mov    eax,0x0 // 1 입력시
0x0000000000400f91 <+62>: jmp    0x400f98 <phase_3+69>
0x0000000000400f93 <+64>: mov    eax,0x208 // 0 입력시
0x0000000000400f98 <+69>: sub    eax,0x6e
0x0000000000400f9b <+72>: jmp    0x400fa2 <phase_3+79>
0x0000000000400f9d <+74>: mov    eax,0x0 // 2 입력시
0x0000000000400fa2 <+79>: add    eax,0x248
0x0000000000400fa7 <+84>: jmp    0x400fae <phase_3+91>
0x0000000000400fa9 <+86>: mov    eax,0x0 // 3 입력시
0x0000000000400fae <+91>: sub    eax,0x26c
0x0000000000400fb3 <+96>: jmp    0x400fba <phase_3+103>
0x0000000000400fb5 <+98>: mov    eax,0x0 // 4 입력시
0x0000000000400fba <+103>: add    eax,0x26c
0x0000000000400fbf <+108>: jmp    0x400fc6 <phase_3+115>
0x0000000000400fc1 <+110>: mov    eax,0x0 // 5 입력시
0x0000000000400fc6 <+115>: sub    eax,0x26c
0x0000000000400fcb <+120>: jmp    0x400fd2 <phase_3+127>
0x0000000000400fcd <+122>: mov    eax,0x0 // 6 입력시
0x0000000000400fd2 <+127>: add    eax,0x26c
0x0000000000400fd7 <+132>: jmp    0x400fde <phase_3+139>
0x0000000000400fd9 <+134>: mov    eax,0x0 // 7 입력시
0x0000000000400fde <+139>: sub    eax,0x26c
0x0000000000400fe3 <+144>: jmp    0x400fef <phase_3+156>
0x0000000000400fe5 <+146>: call   0x401564 <explode_bomb>
0x0000000000400fea <+151>: mov    eax,0x0
0x0000000000400fef <+156>: cmp    DWORD PTR [rsp+0xc],0x5 // 처음 입력값이 5보다 크면 터짐
0x0000000000400ff4 <+161>: jg     0x400ffc <phase_3+169>
0x0000000000400ff6 <+163>: cmp    eax,DWORD PTR [rsp+0x8] // eax == (2번째 입력값)
0x0000000000400ffa <+167>: je     0x401001 <phase_3+174>
0x0000000000400ffc <+169>: call   0x401564 <explode_bomb>
0x0000000000401001 <+174>: add    rsp,0x18
0x0000000000401005 <+178>: ret

```

```

gdb> x/s 0x4032fc
0x4032fc: "%d %d"

```

앞에서 분석했듯이, `sscanf` 함수가 `eax`에 받은 인자의 개수를 판단하니, 1보다 크지 않은 경우는 폭탄이 터진다.

```
gdb> x/20gx 0x402590
0x402590:      0x0000000000400f93      0x0000000000400f8c
0x4025a0:      0x0000000000400f9d      0x0000000000400fa9
0x4025b0:      0x0000000000400fb5      0x0000000000400fc1
0x4025c0:      0x0000000000400fcd      0x0000000000400fd9
...
```

`phase_3` <+50>을 위해 `0x402590`을 읽어보면, 명령어의 주소가 들어 있는 것을 확인할 수 있었다. 만약 첫번째 `0`을 입력하면, `0x0000000000400f93`부터, `1`을 입력하면 `0x0000000000400f8c`부터 ... 이런식으로 명령어가 실행되는 것을 확인할 수 있다.
각 명령어의 주소를 보면, 처음에 `eax`를 `0`으로 `init` 하고, 특정 명령을 수행하는 것을 확인할 수 있었다. 위 함수의 중요 부분을 `c`언어 코드로 작성하면 다음과 같다.

```
phase_3(x, y):
    result = 0
    switch(x):
        case 0: result += 0x208
        case 1: result -= 0x6e
        case 2: result += 0x248
        case 3: result -= 0x26c
        case 4: result += 0x26c
        case 5: result -= 0x26c
        case 6: result += 0x26c
        case 7: result -= 0x26c

    if (result == y) : return true
    else : return false
```

간단한 브루트 포스를 통해, 각 첫번째 입력값에 대한 만족시키는 두번째 입력값을 찾을 수 있었다.

```
i = 0, cnt = 374
i = 1, cnt = -146
i = 2, cnt = -36
i = 3, cnt = -620
i = 4, cnt = 0
i = 5, cnt = -620
i = 6, cnt = 0 // 0x0000000000400fef <+156>에 의해 안 됨.
i = 7, cnt = -620 // 0x0000000000400fef <+156>에 의해 안 됨.
```

5보다 크면 터지므로, `i=0` 부터 `i=5`까지 입력이 가능하다.

0 374

phase_4

```
Dump of assembler code for function phase_4:
0x0000000000401039 <+0>:      sub     rsp,0x18
0x000000000040103d <+4>:      lea     rcx,[rsp+0x8]
0x0000000000401042 <+9>:      lea     rdx,[rsp+0xc]
0x0000000000401047 <+14>:     mov     esi,0x4032fc
0x000000000040104c <+19>:     mov     eax,0x0
0x0000000000401051 <+24>:     call   0x400c30 <__isoc99_sscanf@plt>
0x0000000000401056 <+29>:     cmp     eax,0x2 // 2개의 입력
0x0000000000401059 <+32>:     jne     0x401062 <phase_4+41>
0x000000000040105b <+34>:     cmp     DWORD PTR [rsp+0xc],0xe // 1번째 입력이 0xe 이하여야 함
0x0000000000401060 <+39>:     jbe     0x401067 <phase_4+46>
0x0000000000401062 <+41>:     call   0x401564 <explode_bomb>
0x0000000000401067 <+46>:     mov     edx,0xe
0x000000000040106c <+51>:     mov     esi,0x0
0x0000000000401071 <+56>:     mov     edi,DWORD PTR [rsp+0xc]
0x0000000000401075 <+60>:     call   0x401006 <func4>
0x000000000040107a <+65>:     cmp     eax,0x15 // eax의 값이 0x15이어야 함.
0x000000000040107d <+68>:     jne     0x401086 <phase_4+77>
0x000000000040107f <+70>:     cmp     DWORD PTR [rsp+0x8],0x15 // 2번째 입력이 0x15
0x0000000000401084 <+75>:     je      0x40108b <phase_4+82>
0x0000000000401086 <+77>:     call   0x401564 <explode_bomb>
...
```

`func4`가 수상하다.

- `edi` : (1번째 입력값)
- `esi` : `0x0`

• `edx : 0xe`
위의 인자로 `func4` 함수를 실행한다. 그 결과가 `0x15(21)`가 되어야 하며, 우리의 2번째 입력 값이 `0x15(21)`이 되어야 한다.

```
gdb> x/s 0x4032fc
0x4032fc:      "%d %d"          // 2개의 숫자 입력
```

```
Dump of assembler code for function func4:
0x000000000401006 <+0>:  push    rbx
0x000000000401007 <+1>:  mov     eax,edx
0x000000000401009 <+3>:  sub     eax,esi          // eax = edx - esi
0x00000000040100b <+5>:  mov     ebx,eax          // ebx = edx - esi
0x00000000040100d <+7>:  shr     ebx,0x1f
0x000000000401010 <+10>: add     eax,ebx          // eax += ebx
0x000000000401012 <+12>: sar     eax,1            // eax >> 1 (부호 유지, 2로 나누기)
0x000000000401014 <+14>: lea     ebx,[rax+rsi*1]
// ebx = (edx - esi) / 2 + esi
0x000000000401017 <+17>: cmp     ebx,edi
0x000000000401019 <+19>: jle     0x401027 <func4+33> // ebx <= edi 이면 점프
0x00000000040101b <+21>: lea     edx,[rbx-0x1]     // edx = rbx의 값 - 1
0x00000000040101e <+24>: call    0x401006 <func4>  // 재귀
0x000000000401023 <+29>: add     eax,ebx          // eax += ebx
0x000000000401025 <+31>: jmp     0x401037 <func4+49>
0x000000000401027 <+33>: mov     eax,ebx          // eax = ebx
0x000000000401029 <+35>: cmp     ebx,edi          // ebx >= edi 이면 점프
0x00000000040102b <+37>: jge     0x401037 <func4+49>
0x00000000040102d <+39>: lea     esi,[rbx+0x1]     // esi = rbx의 값 + 1
0x000000000401030 <+42>: call    0x401006 <func4>  // 재귀
0x000000000401035 <+47>: add     eax,ebx          // eax += ebx
...
```

`func4` 함수 안에서 `func4`를 호출하는 것을 볼 수 있으며, 이는 재귀함수의 꼴로 보여진다. 만약 `ebx > edi` 이면 `edx = [rbx-0x1]`로 `func4`를 호출하고, `ebx < edi` 이면 `esi = [rbx+0x1]`로 `func4`로 호출한다. 최종적으로 `ebx == edi` 일 때, 재귀가 마무리되며, 그 동안의 `ebx` 값을 더한 값을 리턴하는 함수임을 알 수 있다. 위 함수를 `c`를 이용해 표현하면 다음과 같다.

```
int func4(int edi, int esi, int edx) // target, left, right
{
    int ebx;
    int rax;                // result

    ebx = (edx - esi) / 2 + esi;    // (left + right) / 2
    if (ebx > edi) {return ebx + func4(edi, esi, ebx - 1);} // target이 작음
    rax = ebx;
    if (ebx < edi) {return ebx + func4(edi, ebx + 1, edx);} // target이 큼
    return rax;
}
```

즉, 이 함수는 처음에 `esi = 0, edx = 0xe`로 설정해 이분 탐색하면서 값을 저장하는 함수이다. 이 함수의 결과값이 `0x15(21)`이 되면 되고, 간단한 브루트 포스를 통해 `edi`의 값을 찾을 수 있었다.

6 21

phase_5

```
Dump of assembler code for function phase_5:
0x000000000401090 <+0>:  push    rbx
0x000000000401091 <+1>:  sub     rsp,0x10
0x000000000401095 <+5>:  mov     rbx,rdi
0x000000000401098 <+8>:  call    0x4012e1 <string_length>
0x00000000040109d <+13>: cmp     eax,0x6          // 6글자 입력
0x0000000004010a0 <+16>: je      0x4010e2 <phase_5+82>
0x0000000004010a2 <+18>: call    0x401564 <explode_bomb>
0x0000000004010a7 <+23>: jmp     0x4010e2 <phase_5+82>
0x0000000004010a9 <+25>: movzx   edx,BYTE PTR [rbx+rax*1]
0x0000000004010ad <+29>: and     edx,0xf          // edx = edx & 0x15 : LSB만 남김.
0x0000000004010b0 <+32>: movzx   edx,BYTE PTR [rdx+0x4025d0] // "maduiersnfotvbyl"[rdx]
0x0000000004010b7 <+39>: mov     BYTE PTR [rsp+rax*1],dl
0x0000000004010ba <+42>: add     rax,0x1
0x0000000004010be <+46>: cmp     rax,0x6          // 6번 반복
0x0000000004010c2 <+50>: jne     0x4010a9 <phase_5+25>
0x0000000004010c4 <+52>: mov     BYTE PTR [rsp+0x6],0x0
0x0000000004010c9 <+57>: mov     esi,0x40257a     // "sarbres"
0x0000000004010ce <+62>: mov     rdi,rsp
0x0000000004010d1 <+65>: call    0x4012fe <strings_not_equal>
0x0000000004010d6 <+70>: test    eax,eax
0x0000000004010d8 <+72>: je      0x4010e9 <phase_5+89>
0x0000000004010da <+74>: call    0x401564 <explode_bomb>
0x0000000004010df <+79>: nop
```

```

0x00000000004010e0 <+80>: jmp 0x4010e9 <phase_5+89>
0x00000000004010e2 <+82>: mov eax,0x0
0x00000000004010e7 <+87>: jmp 0x4010a9 <phase_5+25>
...

```

string length와 string not equal은 위에서 분석했다. 최종적으로 0x40257a에 있는 값과 비교를 한다. <phase_5>+25부터 <phase_5>+50까지 우리가 입력한 문자열인 [rbx]의 값을 1바이트씩 읽어와 0xf와 and 연산을 하고, 0x4025d0에 있는 값의 인덱스를 통해 가져온다.

```

gdb> x/s 0x40257a
0x40257a: "sabres"

```

우리가 입력한 6개의 문자가 특정한 연산을 거쳐서 "sarbres"와 같아지면 된다.

```

gdb> x/s 0x4025d0
0x4025d0: "madiuersnfotvbylWow! You've defused the secret stage!"

```

즉, (입력한 문자) & 0xf -> 0x4025d0 위치의 문자 이고, 그 결과값이 "sabres" 가 되면 된다. 또한 secret stage가 존재하는 것을 확인할 수 있었다.

```

array = "madiuersnfotvbyl" # 대상 문자열
target = 'sabres' # 목표 문자열
result = ''

for i in range(6):
    num = array.find(target[i]) # 대상 문자열의 index를 읽어온다.
    result += chr(num + 0x40) # LSB + 0x40를 계산해 출력가능한 문자열로 한정시킨다. (LSB는 유지므로, & 0xf 값은 동일함.)

print(result)
>> GAMFEG

```

간단한 python 코드로 입력값을 얻을 수 있었다.

phase_6

```

Dump of assembler code for function phase_6:
0x00000000004010ef <+0>: push r13
0x00000000004010f1 <+2>: push r12
0x00000000004010f3 <+4>: push rbp
0x00000000004010f4 <+5>: push rbx
0x00000000004010f5 <+6>: sub rsp,0x58
0x00000000004010f9 <+10>: lea rsi,[rsp+0x30]
0x00000000004010fe <+15>: call 0x40164e <read_six_numbers> // 6개의 숫자
0x0000000000401103 <+20>: lea r13,[rsp+0x30]
0x0000000000401108 <+25>: mov r12d,0x0
0x000000000040110e <+31>: mov rbp,r13 // index 4 증가...
0x0000000000401111 <+34>: mov eax,DWORD PTR [r13+0x0]
0x0000000000401115 <+38>: sub eax,0x1
0x0000000000401118 <+41>: cmp eax,0x5
0x000000000040111b <+44>: jbe 0x401122 <phase_6+51>
0x000000000040111d <+46>: call 0x401564 <explode_bomb>
0x0000000000401122 <+51>: add r12d,0x1
0x0000000000401126 <+55>: cmp r12d,0x6 // r12d != 6 (점프 분기문)
0x000000000040112a <+59>: jne 0x401133 <phase_6+68>
0x000000000040112c <+61>: mov esi,0x0
0x0000000000401131 <+66>: jmp 0x401175 <phase_6+134>
0x0000000000401133 <+68>: mov ebx,r12
0x0000000000401136 <+71>: movsxd rax,ebx
0x0000000000401139 <+74>: mov eax,DWORD PTR [rsp+rax*4+0x30]
// stack을 순회하면서 같은 값이 있는지 확인함.
0x000000000040113d <+78>: cmp DWORD PTR [rbp+0x0],eax // rbp != eax
0x0000000000401140 <+81>: jne 0x401147 <phase_6+88>
0x0000000000401142 <+83>: call 0x401564 <explode_bomb>
0x0000000000401147 <+88>: add ebx,0x1
0x000000000040114a <+91>: cmp ebx,0x5
0x000000000040114d <+94>: jle 0x401136 <phase_6+71>
0x000000000040114f <+96>: add r13,0x4 // index 4 증가
0x0000000000401153 <+100>: jmp 0x40110e <phase_6+31> // 점프
// 만약에 6개의 입력 중에 같은 입력이 존재하면 터짐.
// rsi : [rsp+0x30]
0x0000000000401155 <+102>: mov rdx,QWORD PTR [rdx+0x8]
0x0000000000401159 <+106>: add eax,0x1
0x000000000040115c <+109>: cmp eax,ecx
0x000000000040115e <+111>: jne 0x401155 <phase_6+102>
0x0000000000401160 <+113>: jmp 0x401167 <phase_6+120>
0x0000000000401162 <+115>: mov edx,0x6042f0
0x0000000000401167 <+120>: mov QWORD PTR [rsp+rsi*2],rdx // stack에 노드의 주소값이 담긴다.
0x000000000040116b <+124>: add rsi,0x4

```

```

0x000000000040116f <+128>: cmp     rsi,0x18
0x0000000000401173 <+132>: je      0x40118a <phase_6+155>
0x0000000000401175 <+134>: mov     ecx,DWORD PTR [rsp+rsi*1+0x30]
0x0000000000401179 <+138>: cmp     ecx,0x1
0x000000000040117c <+141>: jle     0x401162 <phase_6+115>
0x000000000040117e <+143>: mov     eax,0x1
0x0000000000401183 <+148>: mov     edx,0x6042f0
0x0000000000401188 <+153>: jmp     0x401155 <phase_6+102>
// stack에 node의 주소값들이 담긴다.
0x000000000040118a <+155>: mov     rbx,QWORD PTR [rsp]
0x000000000040118e <+159>: lea     rax,[rsp+0x8]
0x0000000000401193 <+164>: lea     rsi,[rsp+0x30]
0x0000000000401198 <+169>: mov     rcx,rbx
0x000000000040119b <+172>: mov     rdx,QWORD PTR [rax]
0x000000000040119e <+175>: mov     QWORD PTR [rcx+0x8],rdx // 다음 노드의 주소를 수정
0x00000000004011a2 <+179>: add     rax,0x8
0x00000000004011a6 <+183>: cmp     rax,rsi
0x00000000004011a9 <+186>: je      0x4011b0 <phase_6+193>
0x00000000004011ab <+188>: mov     rcx,rdx
0x00000000004011ae <+191>: jmp     0x40119b <phase_6+172>

0x00000000004011b0 <+193>: mov     QWORD PTR [rdx+0x8],0x0
0x00000000004011b8 <+201>: mov     ebp,0x5
0x00000000004011bd <+206>: mov     rax,QWORD PTR [rbx+0x8] // rax = [rbx+0x8] : 다음 노드의 주소
0x00000000004011c1 <+210>: mov     eax,DWORD PTR [rax] // rax = [[rbx+0x8]] : 다음 노드의 값
0x00000000004011c3 <+212>: cmp     DWORD PTR [rbx],eax
0x00000000004011c5 <+214>: jle     0x4011cc <phase_6+221> // [rbx] >= rax(다음 노드의 값)이 성립해야함.
0x00000000004011c7 <+216>: call    0x401564 <explode_bomb>
0x00000000004011cc <+221>: mov     rbx,QWORD PTR [rbx+0x8]
0x00000000004011d0 <+225>: sub     ebp,0x1
0x00000000004011d3 <+228>: jne     0x4011bd <phase_6+206> // 5번 반복
// 즉, 노드의 값을 작은 순서대로 배열하면 해결할 수 있다. <node6>이 0x5b로 가장 작고
// <node5>, <node2>, <node4>, <node3>, <node1> 순서대로 작다.
...
```

앞에서 봤던 <read_six_numbers>를 통해 입력 값이 stack에 담긴다. 함수가 길어서 판단하기 어려운데, jmp를 기준으로 나눠서 보자.

<phase_6>+100 까지를 살펴보면, 이중 반복문을 사용해, 우리가 입력한 값중에서 같은 값이 있는지 판단하고, 만약 같은 값이 있으면 터지는 구조이다. 그 다음은 stack에 노드의 주소를 읽어오고, 그 노드를 참조해 노드의 값을 읽어온다. 그 후, stack에 있는 노드의 주소를 참조해, 앞의 값이 다음 스택의 값보다 작으면 통과할 수 있다. 따라서, 노드를 작은 순서대로 입력 하면 통과할 수 있을 것이다.

("1 2 3 4 5 6" 입력시)

```

gdb> x/10gx $rsp
0x7fffffff870: 0x00000000006042f0      0x0000000000604300
0x7fffffff880: 0x0000000000604310      0x0000000000604320
0x7fffffff890: 0x0000000000604330      0x0000000000604340
0x7fffffff8a0: 0x0000000200000001      0x0000000400000003
0x7fffffff8b0: 0x0000000600000005      0x000000060047c0
```

- [rsp+0x30] ~ [rsp+0x48]에 우리의 입력이 존재한다.
- [rsp] ~ [rsp+0x28]에 입력한 번호의 노드 주소가 담겨있다.

<phase_6+0>

```

gdb> x/20gx 0x6042f0
0x6042f0 <node1>:      0x00000001000003ca      0x0000000000604300
0x604300 <node2>:      0x000000020000022d      0x0000000000604310
0x604310 <node3>:      0x00000003000002f0      0x0000000000604320
0x604320 <node4>:      0x000000040000025b      0x0000000000604330
0x604330 <node5>:      0x0000000500000087      0x0000000000604340
0x604340 <node6>:      0x000000060000005b      0x0000000000000000
```

("6 5 4 3 2 1" 입력시)

<phase_6+193>

```

gef> x/20gx 0x6042f0
0x6042f0 <node1>:      0x00000001000003ca      0x0000000000604300
0x604300 <node2>:      0x000000020000022d      0x00000000006042f0
0x604310 <node3>:      0x00000003000002f0      0x0000000000604300
0x604320 <node4>:      0x000000040000025b      0x0000000000604310
0x604330 <node5>:      0x0000000500000087      0x0000000000604320
0x604340 <node6>:      0x000000060000005b      0x0000000000604330
```

노드는 다음과 같다. [0x6042f0]에는 노드의 값이, [0x6042f0+0x04]에는 노드의 번호가, [0x6042f0+0x08]에는 다음의 노드 주소가 들어있다. <node_6>의 다음 노드는 <node_5>, <node_5>의 다음 노드는 <node_4> .. 이런 식으로 노드가 재배열된 것을 볼 수 있었다.

앞에서 말했듯, 작은 값을 가지고 있는 노드 순서대로 입력하면 통과할 수 있다.

6 5 2 4 3 1

secret_phase

```
# 0x60479c <num_input_strings>
```

지금까지 입력한 문자열의 갯수가 들어있다. 6개의 phase를 모두 통과하면 들어갈 기회를 가진다.

```
Dump of assembler code for function phase_defused:
0x0000000004017b7 <+0>:    sub    rsp,0x68
0x0000000004017bb <+4>:    mov    edi,0x1
0x0000000004017c0 <+9>:    call   0x4014a0 <send_msg>
0x0000000004017c5 <+14>:   cmp     DwORD_PTR [rip+0x202fd0],0x6
0x0000000004017cc <+21>:   jne     0x40183b <phase_defused+132>
0x0000000004017ce <+23>:   lea     r8,[rsp+0x10]
0x0000000004017d3 <+28>:   lea     rcx,[rsp+0x8]
0x0000000004017d8 <+33>:   lea     rdx,[rsp+0xc]
0x0000000004017dd <+38>:   mov     esi,0x403346 // "%d %d %s"
0x0000000004017e2 <+43>:   mov     edi,0x6048b0 // phase_4에서의 입력
0x0000000004017e7 <+48>:   mov     eax,0x0
0x0000000004017ec <+53>:   call    0x400c30 <__isoc99_sscanf@plt>
0x0000000004017f1 <+58>:   cmp     eax,0x3
0x0000000004017f4 <+61>:   jne     0x401827 <phase_defused+112>
0x0000000004017f6 <+63>:   mov     esi,0x40334f
0x0000000004017fb <+68>:   lea     rdi,[rsp+0x10]
0x000000000401800 <+73>:   call    0x4012fe <strings_not_equal>
0x000000000401805 <+78>:   test    eax,eax
0x000000000401807 <+80>:   jne     0x401827 <phase_defused+112>
0x000000000401809 <+82>:   mov     edi,0x4031b0
0x00000000040180e <+87>:   call    0x400b40 <puts@plt>
0x000000000401813 <+92>:   mov     edi,0x4031d8
0x000000000401818 <+97>:   call    0x400b40 <puts@plt>
0x00000000040181d <+102>:  mov     eax,0x0
0x000000000401822 <+107>:  call    0x40121e <secret_phase> // secret_phase
0x000000000401827 <+112>:  mov     edi,0x403210
0x00000000040182c <+117>:  call    0x400b40 <puts@plt>
0x000000000401831 <+122>:  mov     edi,0x403240
0x000000000401836 <+127>:  call    0x400b40 <puts@plt>
0x00000000040183b <+132>:  add     rsp,0x68
0x00000000040183f <+136>:  ret
```

```
gdb> x/s 0x403346
0x403346:      "%d %d %s"
gdb> x/s 0x6048b0
0x6048b0 <input_strings+240>:  "6 21"
gdb> x/s 0x40334f
0x40334f:      "SecretPhase"
```

0x403346를 보면 2개의 정수와 1개의 문자열을 받고, 0x6048b0를 확인하면, 우리가 phase 4에서 사용했던 입력이 있는 것을 알 수 있다. 만약에 우리가 "6 21 SecretPhase"를 입력한다면, phase_6을 통과한 이후에 secret_phase로 들어 갈 수 있을 것이다.

```
phase_4 : "6 21 SecretPhase"
```

```
Dump of assembler code for function secret_phase:
0x00000000040121e <+0>:    push    rbx
0x00000000040121f <+1>:    call    0x401691 <read_line>
0x000000000401224 <+6>:    mov     edx,0xa
0x000000000401229 <+11>:   mov     esi,0x0
0x00000000040122e <+16>:   mov     rdi,rax
0x000000000401231 <+19>:   call    0x400c00 <strtol@plt>
0x000000000401236 <+24>:   mov     rbx,rax
0x000000000401239 <+27>:   lea     eax,[rax-0x1]
0x00000000040123c <+30>:   cmp     eax,0x3e8
0x000000000401241 <+35>:   jbe     0x401248 <secret_phase+42>
0x000000000401243 <+37>:   call    0x401564 <explode_bomb>
0x000000000401248 <+42>:   mov     esi,ebx // (입력값)
0x00000000040124a <+44>:   mov     edi,0x604110 // <n1> ...
0x00000000040124f <+49>:   call    0x4011e0 <fun7> // fun7의 결과값이 2이어야 함.
0x000000000401254 <+54>:   cmp     eax,0x2
0x000000000401257 <+57>:   je      0x40125e <secret_phase+64>
0x000000000401259 <+59>:   call    0x401564 <explode_bomb>
...
```

- edi = 0x604110
 - esi = (유저의 입력값)
- 위의 인자들로 fun7 함수를 호출한다. fun7의 값이 0x2이라면 성공적으로 secret_phase도 해결할 수 있을 것이다.


```

Dump of assembler code for function fun7:
0x0000000004011e0 <+0>:      sub     rsp,0x8
0x0000000004011e4 <+4>:      test    rdi,rdi                // rdi == 0 이면 return -1
0x0000000004011e7 <+7>:      je      0x401214 <fun7+52>
0x0000000004011e9 <+9>:      mov     edx,DWORD PTR [rdi]
0x0000000004011eb <+11>:     cmp     edx,esi                // edx <= esi 이면 점프
0x0000000004011ed <+13>:     jle     0x4011fc <fun7+28>
0x0000000004011ef <+15>:     mov     rdi,QWORD PTR [rdi+0x8] // 왼쪽 노드
0x0000000004011f3 <+19>:     call    0x4011e0 <fun7>
0x0000000004011f8 <+24>:     add     eax,eax                // eax = 2 * eax
0x0000000004011fa <+26>:     jmp     0x401219 <fun7+57>
0x0000000004011fc <+28>:     mov     eax,0x0
0x000000000401201 <+33>:     cmp     edx,esi                // edx == esi 이면 종료
0x000000000401203 <+35>:     je      0x401219 <fun7+57>
0x000000000401205 <+37>:     mov     rdi,QWORD PTR [rdi+0x10] // 오른쪽 노드
0x000000000401209 <+41>:     call    0x4011e0 <fun7>
0x00000000040120e <+46>:     lea     eax,[rax+rax*1+0x1]    // eax = 2 * eax + 1
0x000000000401212 <+50>:     jmp     0x401219 <fun7+57>
0x000000000401214 <+52>:     mov     eax,0xffffffff        // return -1
...

```

fun7 안에서 fun7을 부르는 재귀함수의 끝을 하고 있는데, `edx == esi` 이면 `eax`를 0을 반환해 종료하고, 아니면 재귀 탐색을 하고, 만약 더 이상 값이 없으면, -1을 반환한다.

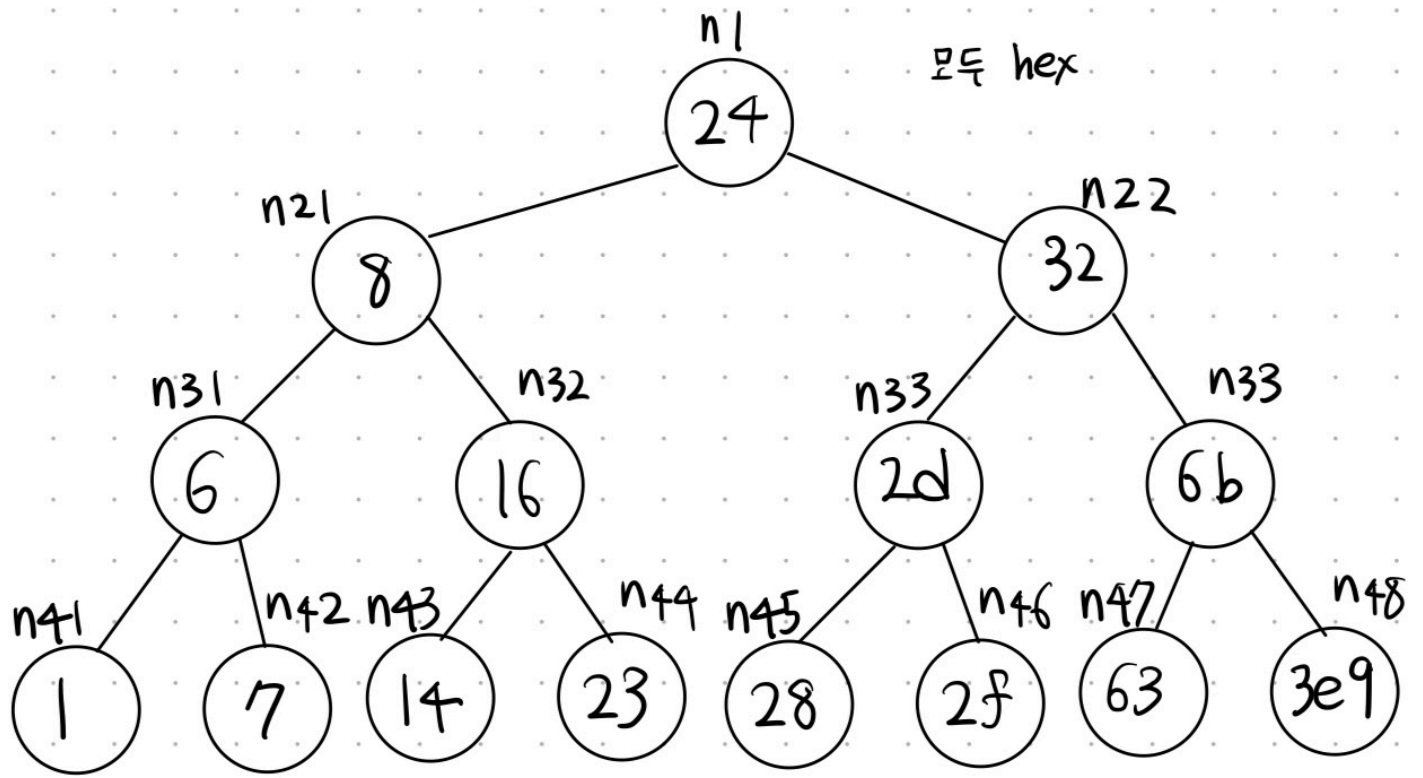
- `edi == 0` : return -1
- `edx > esi` : return `2 * fun7([edi + 0x8], esi)`
- `edx < esi` : return `2 * func7([edi + 0x10], esi) + 1`
- `edx == esi` : return 0

```

gdb> x/80gx 0x604110
0x604110 <n1>:      0x0000000000000024      0x00000000000604130
0x604120 <n1+16>:    0x00000000000604150      0x0000000000000000
0x604130 <n21>:      0x0000000000000008      0x000000000006041b0
0x604140 <n21+16>:   0x00000000000604170      0x0000000000000000
0x604150 <n22>:      0x0000000000000032      0x00000000000604190
0x604160 <n22+16>:   0x000000000006041d0      0x0000000000000000
0x604170 <n32>:      0x0000000000000016      0x00000000000604290
0x604180 <n32+16>:   0x00000000000604250      0x0000000000000000
0x604190 <n33>:      0x000000000000002d      0x000000000006041f0
0x6041a0 <n33+16>:   0x000000000006042b0      0x0000000000000000
0x6041b0 <n31>:      0x0000000000000006      0x00000000000604210
0x6041c0 <n31+16>:   0x00000000000604270      0x0000000000000000
0x6041d0 <n34>:      0x000000000000006b      0x00000000000604230
0x6041e0 <n34+16>:   0x000000000006042d0      0x0000000000000000
0x6041f0 <n45>:      0x0000000000000028      0x0000000000000000
0x604200 <n45+16>:   0x0000000000000000      0x0000000000000000
0x604210 <n41>:      0x0000000000000001      0x0000000000000000
0x604220 <n41+16>:   0x0000000000000000      0x0000000000000000
0x604230 <n47>:      0x0000000000000063      0x0000000000000000
0x604240 <n47+16>:   0x0000000000000000      0x0000000000000000
0x604250 <n44>:      0x0000000000000023      0x0000000000000000
0x604260 <n44+16>:   0x0000000000000000      0x0000000000000000
0x604270 <n42>:      0x0000000000000007      0x0000000000000000
0x604280 <n42+16>:   0x0000000000000000      0x0000000000000000
0x604290 <n43>:      0x0000000000000014      0x0000000000000000
0x6042a0 <n43+16>:   0x0000000000000000      0x0000000000000000
0x6042b0 <n46>:      0x000000000000002f      0x0000000000000000
0x6042c0 <n46+16>:   0x0000000000000000      0x0000000000000000
0x6042d0 <n48>:      0x00000000000003e9      0x0000000000000000
0x6042e0 <n48+16>:   0x0000000000000000      0x0000000000000000

```

자세히 들여다 보면, [0x604110]에는 값, [0x604110+0x8]과 [0x604110+0x10]에는 주소값이 들어있는 것을 알 수 있다. [0x604110+0x8]를 왼쪽 노드, [0x604110+0x10]을 오른쪽 노드라 하면 다음과 같이 BST로 그림을 그릴 수 있다.



메모리의 BST를 구현하고, fun7을 구현해 간단한 브루트 포스로 우리가 원하는 입력값을 구해낼 수 있었다.

```

bst_values = [0x24, 0x8, 0x32, 0x6, 0x16, 0x2d, 0x6b, 0x1, 0x7, 0x14, 0x23, 0x2f, 0x63, 0x3e9]

bst_node = [[1, 2], [3, 4], [5, 6], [7, 8], [9, 10], [11, 12], [13, 14], [-1, -1], [-1, -1], [-1, -1], [-1, -1], [-1, -1], [-1, -1], [-1, -1], [-1, -1]]

def fun7(search_value : int, target_node : int) -> int:
    if (target_node == -1):
        return -1
    if (bst_values[target_node] > search_value):
        return 2 * fun7(search_value, bst_node[target_node][0])
    result = 0
    if (bst_values[target_node] != search_value) :
        return 2 * fun7(search_value, bst_node[target_node][1]) + 1
    return result

for i in range(0x3e9):
    result = fun7(i, 0)
    if result == 0x2:
        print(i, hex(i))

>> 20, 0x14
>> 22, 0x16
  
```

위와 같은 코드로 결과값이 0x2가 되는 입력을 찾을 수 있었다.